

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (11/21)

- Consider a program with two threads running on separate cores in a **dual-core shared-memory multiprocessor**. Assume that all memory contents are **initially 0**:

```
# thread A
A0:  ori  $t0, $zero, 0x4000
A1:  addi $s0, $zero, 1
A2:  sw   $s0, 0($t0)           # (0x4000) <- 1
A3:  lw   $s1, 4($t0)
A4:  addi $s1, $s1, 1
A5:  sw   $s1, 8($t0)           # (0x4008) <- (0x4004) + 1
...

# thread B
B0:  ori  $t0, $zero, 0x4000
B1:  lw   $s2, 0($t0)
B2:  addi $s2, $s2, 1
B3:  sw   $s2, 4($t0)           # (0x4004) <- (0x4000) + 1
B4:  lw   $s3, 8($t0)
B5:  addi $s3, $s3, 1
B6:  sw   $s3, 12($t0)          # (0x400C) <- (0x4008) + 1
...
```

Assume that each core has a **private data cache** with 16-byte blocks. Each cache is **write-back, write-allocate** and does **not** support cache coherence. Assuming both caches are initially empty, you run the program and observe that the memory operations are performed in the following sequence (time→):

B1, A2, B3, A3, A5, B4, B6, A evicts block 0x4000, B evicts block 0x4000

Fill in the table below, specifying the contents of each data word in block 0x4000 in **main memory**, **after** this sequence has completed.

Main Memory	
Address	Data Word Value (in decimal)
0x4000	0 (from thread B)
0x4004	1 (from thread B)
0x4008	0 (from thread B)
0x400C	1 (from thread B)

2. Consider a program with four threads running on separate cores in a **quad-core shared-memory multiprocessor**. Assume that all memory contents are **initially 0**, and there are no private data caches in this system:

```
# thread A
A0: lui $t0, 0xC000
A1: addi $t7, $zero, 1
A2: sw $t7, 0($t0)      # (0xC0000000) <- 1

# thread B
B0: lui $t1, 0xC000
B1: ori $t1, $t1, 0x1000
B2: addi $t7, $zero, 1
B3: sw $t7, 0($t1)      # (0xC0001000) <- 1

# thread C
C0: lui $t1, 0xC000
C1: add $t0, $zero, $t1
C2: ori $t1, $t1, 0x1000
C3: lw $s0, 0($t0)      # [$s0] <- (0xC0000000)
C4: lw $s1, 0($t1)      # [$s1] <- (0xC0001000)

# thread D
D0: lui $t0, 0xC000
D1: add $t1, $zero, $t0
D2: ori $t1, $t1, 0x1000
D3: lw $s2, 0($t1)      # [$s2] <- (0xC0001000)
D4: lw $s3, 0($t0)      # [$s3] <- (0xC0000000)
```

- a) Fill in the table below, specifying whether or not each program outcome (i.e., set of register contents after executing all threads) is **possible** if the multiprocessor enforces **sequential consistency**.

Register Contents After Execution		Possible With Sequential Consistency (Y/N)? (One possible answer listed)
Thread C	Thread D	
[\$s0] = 0 [\$s1] = 0	[\$s2] = 0 [\$s3] = 0	Y: C3, C4, D3, D4, ...
[\$s0] = 0 [\$s1] = 0	[\$s2] = 0 [\$s3] = 1	Y: C3, C4, D3, A2, B4, ...
[\$s0] = 0 [\$s1] = 1	[\$s2] = 0 [\$s3] = 1	Y: C3, D3, B3, C4, A2, D4
[\$s0] = 1 [\$s1] = 0	[\$s2] = 0 [\$s3] = 0	Y: D3, D4, A2, C3, C4, ...
[\$s0] = 1 [\$s1] = 0	[\$s2] = 1 [\$s3] = 0	N: SC requires \$s0 load before \$s1, and \$s2 load before \$s3. Thus it's not possible for first loads to be 1 (meaning A2 & B3 have executed) and 2 nd loads to be 0

$[\$s0] = 1$ $[\$s1] = 0$	$[\$s2] = 0$ $[\$s3] = 1$	Y: D3, A2, D4, C3, C4, ...
$[\$s0] = 1$ $[\$s1] = 1$	$[\$s2] = 0$ $[\$s3] = 0$	Y: D3, D4, A2, B3, C3, C4
$[\$s0] = 1$ $[\$s1] = 1$	$[\$s2] = 0$ $[\$s3] = 1$	Y: D3, A2, B3, D4, C3, C4
$[\$s0] = 1$ $[\$s1] = 1$	$[\$s2] = 1$ $[\$s3] = 0$	Y: B3, D3, D4, A2, C3, C4

- b) Assume that your multiprocessor does **not** enforce sequential consistency. Each of the cores is a **two-wide superscalar MIPS R10K-like** processor. The pipeline diagram for one possible execution of threads C and D is shown below. Loads are performed in X.

insn\cycle	1	2	3	4	5	6	7	8	9	10	11	12	13
C0	F	De	Di	S	X	C	R						
C1	F	De	Di	S	S	S	X	C	R				
C2		F	De	Di	S	S	X	C	R				
C3		F	De	Di	S	S	S	S	X	C	R		
C4			F	De	Di	S	S	S	X	C	R		
D0	F	De	Di	S	X	C	R						
D1	F	De	Di	S	S	S	X	C	R				
D2		F	De	Di	S	S	S	S	X	C	R		
D3		F	De	Di	S	S	S	S	S	S	X	C	R
D4			F	De	Di	S	X	C	R	R	R	R	R

Given this specific execution, fill in the blanks below:

If A2 is performed at cycle _____ and B3 is performed at cycle _____, the program outcome would **not be sequentially consistent**.

The **register contents** at the end of the program would be:

$[\$s0] = \underline{\hspace{2cm}}$, $[\$s1] = \underline{\hspace{2cm}}$, $[\$s2] = \underline{\hspace{2cm}}$, $[\$s3] = \underline{\hspace{2cm}}$.

Answer:

If A2 is performed at cycle 8 and B3 is performed at cycle 10, the program outcome would **not be sequentially consistent**.

The **register contents** at the end of the program would be:

$[\$s0] = \underline{1}$, $[\$s1] = \underline{0}$, $[\$s2] = \underline{1}$, $[\$s3] = \underline{0}$.